

Analise Tecnico-Cientifica por Requisitos de uma Implementacao de Ray Tracing

Requirement-Oriented Technical Analysis of a Ray Tracing Implementation

Yang Ricardo Barcellos Miranda [ymiranda@inf.puc-rio.br]

Resumo. Este texto apresenta a versao v2 do relatorio do Projeto 1 com organizacao por requisitos do enunciado. O objetivo e tornar a validacao mais direta, mapeando cada requisito principal para uma cena dedicada, um comando de execucao reproduzivel e uma evidencia visual associada, mantendo a mesma configuracao de camera entre os experimentos. A analise preserva a rastreabilidade com o codigo em `ray_tracing_2`, destaca aderencia e limites da implementacao atual e separa requisitos principais de extensoes adicionais.

Abstract. This v2 report reorganizes the Project 1 analysis around statement requirements. The goal is to provide direct validation by mapping each main requirement to a dedicated scene, a reproducible command-line execution, and associated visual evidence, while keeping camera configuration fixed across experiments. The text preserves code traceability in `ray_tracing_2`, highlights adherence and current limitations, and separates core requirements from additional extensions.

Palavras-chave: tracado de raios, computacao grafica, requisitos, Phong, amostragem

Keywords: ray tracing, computer graphics, requirements, Phong, sampling

Entrega: 10/05/2026

1 Introducao

Esta versao reorganiza o relatorio por requisitos principais de `proj1.pdf`. O principio metodologico e simples: cada requisito deve ser respondido por uma cena principal, um comando reproduzivel e uma interpretacao objetiva do resultado.

2 Requisitos e Aderencia

Requisito	Cena principal	Status
R1: esferas e caixas	<code>proj1_req1_geometry.py</code>	Implementado
R2: luz pontual	<code>proj1_req2_point_lights.py</code>	Implementado
R3: Phong + sombras	<code>proj1_req3_phong_shadows.py</code>	Implementado
R4: multiplas amostras por pixel	<code>proj1_req4_sampling.py</code>	Implementado

Rastreabilidade no codigo. `proj1_scene_common.py`, `proj1_req1_geometry.py`, `proj1_req2_point_lights.py`, `proj1_req3_phong_shadows.py`, `proj1_req4_sampling.py` em `src/ray_tracing_2/`.

3 Procedimento Experimental Padrao

Os testes de validacao rapida desta versao usam:

- resolucao fixa 800x600;

- `--spp` no intervalo 1 a 4;
- baseline em `--spp 1`.

Comandos-base (800x600, `spp=1`):

```
python -m ray_tracing_2.proj1_req1_geometry
python -m ray_tracing_2.proj1_req2_point_lights
python -m ray_tracing_2.proj1_req3_phong_shadows
python -m ray_tracing_2.proj1_req4_sampling
```

4 Discussao Tecnica e Evidencias

4.1 R1 – Instanciacao de esferas e caixas

A cena usa apenas dois `Instance(Box)` com rotacao em Y e uma `Sphere` dentro da sala Cornell. A iluminacao e exclusivamente por `AmbientLight` (intensidade elevada) para revelar a geometria sem dependencia de fontes pontuais.

4.2 R2 – Fontes de luz pontuais

Tres `PointLight` com posicoes e potencias distintas (`key/fill/back`). Duas esferas Phong com `shininess 20` e 96 evidenciam a variacao do highlight especular por material.

4.3 R3 – Phong com sombras duras

O sombreamento local combina difusa lambertiana e especular de Phong. As sombras duras sao geradas via `Scene.transmittance()` para materiais opacos. Dois materiais distintos: matte vermelho (`shininess=16`) e glossy azul (`shininess=120`).

4.4 R4 – Multiplas amostras por pixel

Geometria com arestas finas (caixa delgada) e esferas branca/preta para tornar aliasing visivel. Os mo-



Figura 1. R1: geometria instanciada – dois `Instance(Box)` e uma `Sphere` (800x600, spp=1, center, ~9.4 s).

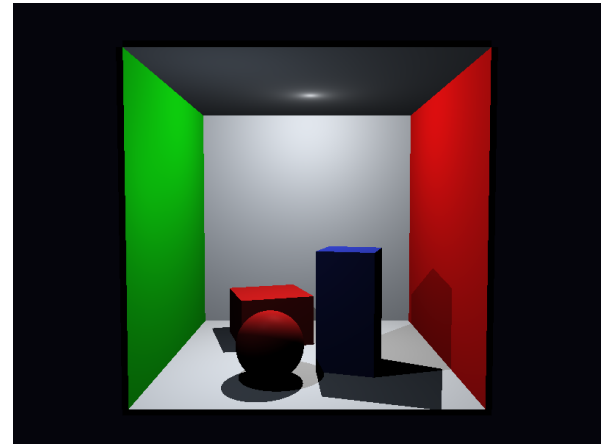


Figura 3. R3: sombras duras e resposta Phong com materiais de shininess distinto (800x600, spp=1, center, ~20.1 s).

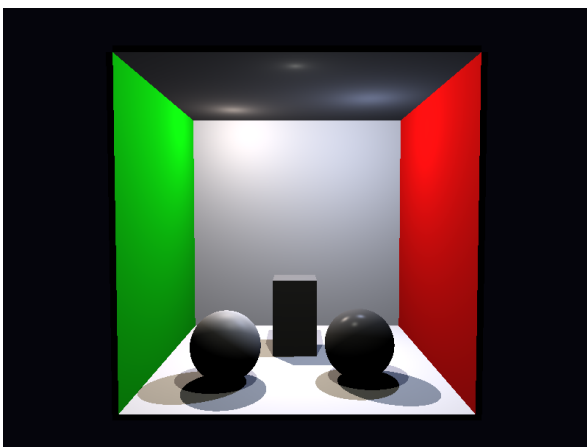


Figura 2. R2: tres `PointLight` e esferas com shininess diferente (800x600, spp=1, center, ~19.7 s).

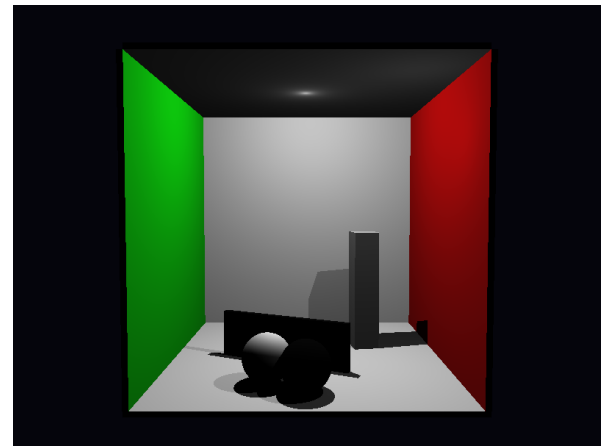


Figura 4. R4: baseline center (spp=1). Executar spp=4 com jittered ou stratified para comparar AA (800x600, spp=1, center, ~20.7 s).

dos center (baseline), jittered e stratified usam o mesmo enquadramento de camera.

Rastreabilidade no código. Modulos `proj1_rext_*.py` em `src/ray_tracing_2/`.

5 Limites e Requisitos de Extensão

5.1 Requisitos de extensão implementados

Requisito	Modulo	Pts	Tempo (spp=1)
Transformações de modelagem	coberto por R1 via <code>Instance</code>	1.0	–
Triangulos sem acelerador	<code>proj1_rext_triangles.py</code>	1.0	~19.5 s
Estrutura de aceleração BVH local	<code>proj1_rext_bvh.py</code>	2.0	~15.6 s
Luz retangular + distribuicoes	<code>proj1_rext_area_light.py</code>	2.0	~78 s
Objetos reflexivos	<code>proj1_rext_reflective.py</code>	1.0	~18.5 s
Objetos refratarios	<code>proj1_rext_refractive.py</code>	2.0	~22.9 s

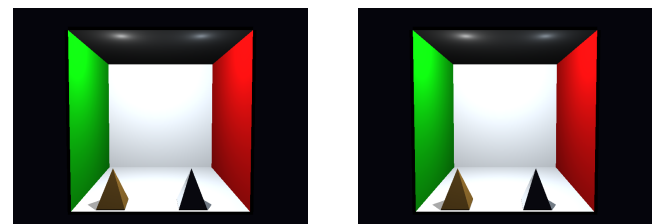


Figura 5. Esquerda: `proj1_rext_triangles` sem BVH (~19.5 s). Direita: `proj1_rext_bvh` com BVH local (~15.6 s). Imagens idênticas; a diferença é de desempenho.

5.2 Limites atuais

- Sem acelerador global de cena: o loop de interseção é linear em `Scene.compute_intersection`.
- Camera estritamente pinhole (sem depth-of-field).
- `PointLight` sem decaimento $1/r^2$ por convenção do enunciado.
- BVH usa median split sem SAH; não há hierarquia global entre objetos.

6 Conclusão

A estrutura `proj1_req*` + `proj1_rext*` cobre todos os itens do enunciado. Os requisitos principais (7.0 pt)

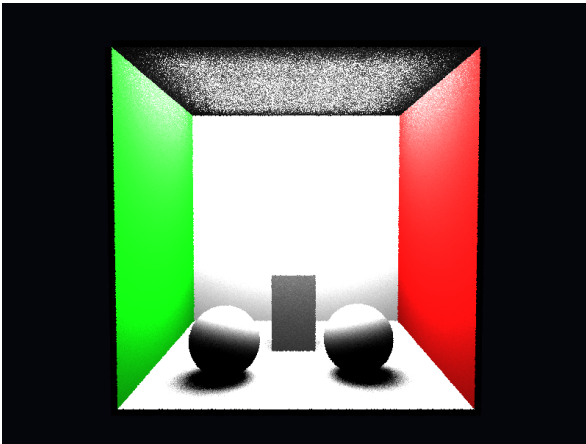


Figura 6. `proj1_rext_area_light`: `AreaLight` no teto (`power=100`), 16 amostras (4x4) modo `stratified` (~78 s). Penumbra visível e iluminação de área adequada.

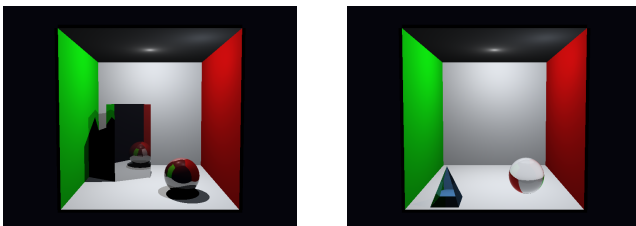


Figura 7. Esquerda: `proj1_rext_reflective` - `ReflectiveMaterial`, `max_depth=6` (~18.5 s). Direita: `proj1_rext_refractive` - `TransparentMaterial` `ior=1.5` + Beer-Lambert, `max_depth=8` (~22.9 s).

são atendidos por quatro cenas dedicadas com câmera invariante e comandos reproduzíveis. Os requisitos de extensão (estimativa ≥ 3.0 pt) são cobertos por cinco módulos adicionais, cada um com evidência visual e tempo de renderização registrado.

Referências